Mapping the Dance of the Stars: Unveiling the Dynamics of N-Body Simulations

Gauss T. Chang[†]

[†] *Department of Physics, National Taiwan University, Taipei, Taiwan,*

b09501028@ntu.edu.tw

December 17, 2024

Abstract

We simulate gravitational N-body systems with elastic collisions in C++ and Fortran. We compare direct pairwise gravity with the Barnes–Hut approximation and measure the effect of OpenMP on the C++ tree traversal. Kinetic energy, potential energy, and angular momentum provide numerical checks across the solvers. The timing experiments use identical initial conditions and separate the speedup from the octree algorithm from the additional speedup from threading.

I Introduction

The N-body problem, which seeks to determine the motions of N distinct masses under mutual gravitational influence, has long been central to our understanding of celestial mechanics and astrophysical systems. While closed-form solutions are limited to special cases, computational simulations now serve as essential tools for studying the evolution of complex systems such as star clusters, planetary systems, and galaxies.

Despite its importance, the N-body problem poses significant computational and numerical challenges. The complexity of direct pairwise force evaluations scales as $O(N^2)$, becoming prohibitively expensive for large N. Additionally, the need to maintain physical fidelity—such as conserving energy and angular momentum—demands careful selection of numerical integration methods and collision-handling procedures.

This article focuses on a computational approach that harnesses both the classical laws governing gravitational interaction and modern ap-

proximation techniques to accelerate computations. I implement and test a Barnes–Hut algorithm to reduce computational complexity to approximately $O(N \log N)$, and I monitor key invariants such as total energy and angular momentum to assess the quality of the simulation. Furthermore, I explore performance improvements and discuss approaches to scaling these methods to larger systems.

In the following sections, I detail the theoretical foundations and algorithms employed (Section II), describe the simulation setup (Section III), present the results, and evaluate the performance and fidelity of my approach.

II Algorithm

In order to model the gravitational interactions among N bodies, I start from fundamental physical laws and incorporate efficient computational techniques. This section first reviews Newton's Law of Gravitation, the force model at the core of my simulation, and then introduces the Barnes–Hut algorithm, which significantly reduces the computational burden for large N.

II.I Newton's Law of Gravitation

The gravitational force between two point masses m_1 and m_2 separated by a distance vector $\mathbf{r} = \mathbf{r}_1 - \mathbf{r}_2$ is given by

$$\mathbf{F} = -G\frac{m_1 m_2}{|\mathbf{r}|^3}\mathbf{r}, \tag{1}$$

where G is the gravitational constant. This force is attractive and acts along the line connecting the two masses. In an N-body system, each particle experiences forces from every other particle, leading to a set of coupled, nonlinear ordinary differential equations. For large N, evaluating each pairwise force directly leads to an $O(N^2)$ complexity, a severe limitation for computational studies of large-scale structures.

To handle dynamical evolution, we employ numerical integration methods. In this study, we utilize time-stepping integrators that update particle velocities and positions incrementally. Ensuring that these integrators preserve essential physical invariants, such as total energy and angular momentum, is key to obtaining meaningful and stable long-term results.

II.II Barnes–Hut Algorithm[1]

The Barnes–Hut algorithm addresses the scaling issue inherent in direct N-body simulations. Rather than evaluating every particle-particle interaction, the algorithm builds a hierarchical tree structure to group particles that are sufficiently distant into a single effective mass at their combined center of mass. By treating distant clusters of particles as single entities, the force calculation for a given particle becomes more efficient, reducing the typical complexity to $O(N \log N)$.

The core idea is to recursively subdivide the computational domain into cells (quadrants in 2D or octants in 3D), constructing a tree that

represents the spatial distribution of particles. Each internal node of the tree corresponds to a cell containing a group of particles, characterized by their total mass and center of mass. During the force calculation phase, if a node is sufficiently far away relative to its size, it can be used as a single approximation source. Otherwise, the tree is traversed further down to obtain finer-grained information.

By carefully tuning parameters such as the opening angle θ, one can achieve a balance between accuracy and performance. A smaller θ yields more accurate results at the cost of additional computations, whereas a larger θ trades off accuracy for speed.

II.III OpenMP Tree Traversal

Once the tree is built, the gravitational acceleration of each target particle can be evaluated independently. In C++, an OpenMP worksharing loop assigns target particles to threads using a static schedule. Each iteration reads the same tree, writes only the target's acceleration, and contributes to a reduction of the gravitational potential energy[2]. The tree build, collision handling, and time integration execute serially. Collision impulses retain their pair order because two contacts can share a particle.

II.IV Time Integration

The simulation uses semi-implicit Euler integration. For a time step Δt, the code first updates the velocity using the current acceleration, then advances the position with that updated velocity:

$$\mathbf{v}(t+\Delta t) = \mathbf{v}(t) + \mathbf{a}(t)\,\Delta t, \tag{2}$$

$$\mathbf{x}(t+\Delta t) = \mathbf{x}(t) + \mathbf{v}(t+\Delta t)\,\Delta t. \tag{3}$$

Here $\mathbf{v}(t)$ includes any collision impulse applied before the step. The method is first-order accurate in Δt. Higher-order methods, such as velocity Verlet, are a possible extension for studying energy behavior over longer intervals.

II.V Elastic Collision

In my simulation, particles are modeled as spherical masses that can collide elastically. A collision occurs when the distance between two particles a and b is less than or equal to the sum of their radii:

$$|\mathbf{x}_a - \mathbf{x}_b| \le (r_a + r_b) \tag{4}$$

Once a collision is detected, we resolve it by treating the encounter as a one-dimensional elastic collision along the line connecting the particle centers. Define the unit normal vector:

$$\mathbf{n} = \frac{\mathbf{x}_a - \mathbf{x}_b}{|\mathbf{x}_a - \mathbf{x}_b|} \tag{5}$$

The relative velocity of the particles along this normal direction is:

$$v_{\mathrm{rel},n} = (\mathbf{v}_a - \mathbf{v}_b) \cdot \mathbf{n}. \tag{6}$$

If $v_{\mathrm{rel},n}$ is positive, the particles are already moving apart, and no collision response is needed. Otherwise, we compute an impulse J that, when

applied equally and oppositely to the two particles, conserves both momentum and kinetic energy. For masses m_a and m_b, the magnitude of the impulse is:

$$J = \frac{2\, v_{\mathrm{rel},n}}{\frac{1}{m_a} + \frac{1}{m_b}}. \tag{7}$$

The post-collision velocities are then updated as follows:

$$\mathbf{v}_a' = \mathbf{v}_a - \frac{J}{m_a}\mathbf{n}, \;\; \mathbf{v}_b' \qquad = \mathbf{v}_b + \frac{J}{m_b}\mathbf{n}. \tag{8}$$

This formulation ensures that the collision is perfectly elastic. The impulse conserves kinetic energy and linear momentum. The position update then carries the bodies apart according to their post-collision velocities.

III Simulation Setup

For my simulations, we begin by generating an initial distribution of N particles placed within a cubic volume of side length L. Each particle is assigned:

- **Mass:** Chosen randomly between 1 and 2, ensuring a modest mass contrast.
- **Position:** Initialized from a uniform distribution within the domain

$$-\frac{L}{2} \leq x, y, z \leq \frac{L}{2}$$

- **Velocity:** Drawn from a small uniform range (e.g., -1 to 1), providing a non-zero initial kinetic state that allows the system to evolve dynamically.
- **Radius:** Assigned between 0.001 and 0.05, enabling detection and handling of elastic collisions when particles come into close proximity.

These initial conditions, as illustrated in the project slides, serve as a simple yet flexible baseline for exploring gravitational dynamics. The time domain is discretized into steps of size Δt. At each timestep, we:

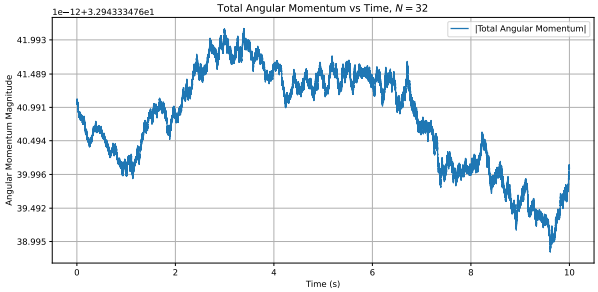
1. **Build the Barnes–Hut tree:** Particles are sorted into a hierarchical structure for efficient force approximations.
2. **Compute Forces:** Gravitational forces on each particle are derived using the Barnes–Hut approximation. Close pairs are treated at a finer resolution, while distant groups of particles are represented by their center of mass.
3. **Handle Collisions:** If two particles overlap (based on their radii), we resolve their encounter with an elastic collision model, preserving linear momentum and ensuring a physically plausible outcome.
4. **Update Kinematics:** Particle positions and velocities are integrated forward in time using a chosen time integrator. We record total energy and angular momentum after each update to monitor numerical fidelity.

By proceeding in this manner, we can visualize the initial random configurations evolv-

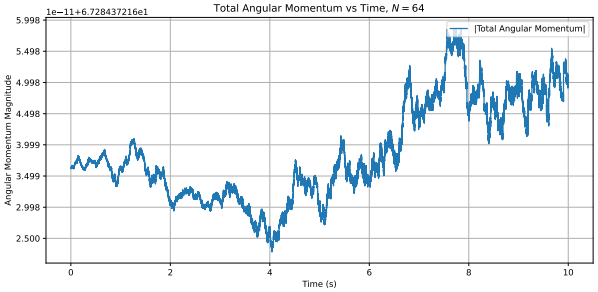
ing into complex structures, as demonstrated in my slide presentations and supplementary animations. These setups, while simplified, are an effective proving ground for testing algorithmic performance, verifying energy and momentum conservation, and identifying numerical drift in large-N simulations.

IV Results

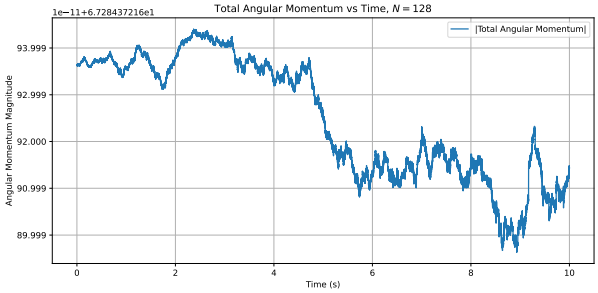
IV.I Angular Momentum



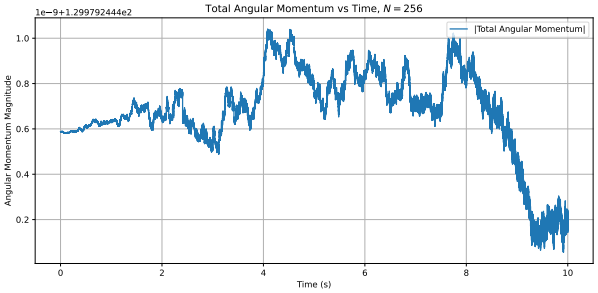
(a) Angular momentum v.s. time, $N = 32$



(b) Angular momentum v.s. time, $N = 64$



(c) Angular momentum v.s. time, $N = 128$



(d) Angular momentum v.s. time, $N = 256$

Figure 1: Angular momentum v.s. time for different particle number.

Angular momentum is a critical conserved quantity in gravitational systems, reflecting the rotational invariants of the system's motion. In principle, a well-posed N-body simulation with accurate numerical integration and appropriate timestep selection should preserve total angular momentum over time.

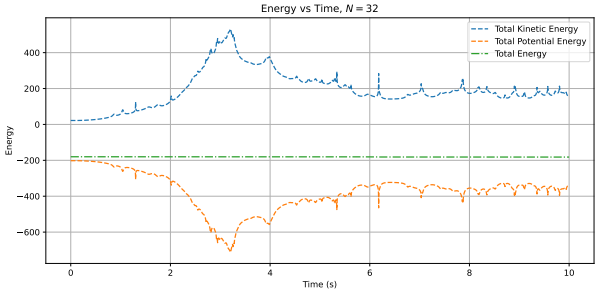
Figures 1a–1d show the evolution of the total angular momentum magnitude as a function of time for simulations with $N = 32, 64, 128$, and 256 particles. Overall, the angular momentum remains remarkably stable throughout the simulations. Although minor fluctuations are present, it is important to consider them in the context of the scale of the system's total angular momentum.

Our simulations maintain a total angular momentum on the order of 10^1, and the observed fluctuations are on the order of 10^{-9} to 10^{-11}. In other words, the relative change in angular momentum is extremely small, on the order of 10^{-10} relative to the magnitude of the total angular momentum. Such tiny deviations are well within acceptable bounds for numerical simulations involving floating-point arithmetic and are likely attributable to truncation errors, round-off errors, and other numerical artifacts inherent in large-scale computations.

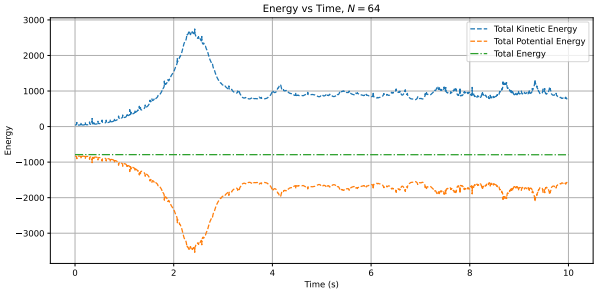
Unlike total energy, which can drift more significantly under certain conditions (especially at larger N or after numerous collision events), angular momentum appears robust and shows no trend of systematic loss or gain over time. This stability suggests that our choice of integration scheme, timestep, and collision handling mechanism effectively preserves rotational invariants. Further refinement of the integration methods or employing adaptive time-stepping could reduce these minuscule fluctuations even more, but for the purposes of this study, the current level of angular momentum conservation is fully satisfactory.

In conclusion, the angular momentum results confirm that the numerical approach and algorithms implemented here are both stable and reliable, providing a solid foundation for exploring more complex gravitational systems.

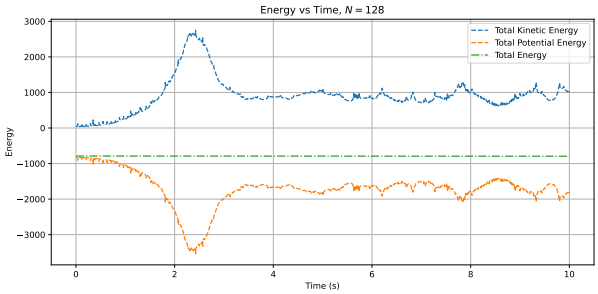
IV.II Total Energy



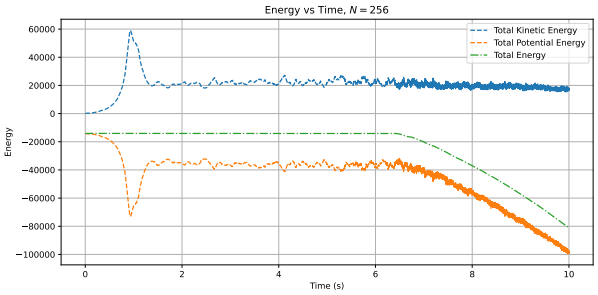
(a) Total energy v.s. time, $N = 32$



(b) Total energy v.s. time, $N = 64$



(c) Total energy v.s. time, $N = 128$



(d) Total energy v.s. time, $N = 256$

Figure 2: Total energy v.s. time for different particle number.

Conservation of total energy, $E_{\text{total}} = E_{\text{kinetic}} + E_{\text{potential}}$, is a stringent test of the numerical fidelity of N-body simulations. In principle, the gravitational N-body system should conserve its total energy if integrated perfectly and if collisions are handled in a physically consistent manner. However, numerical approximations, finite timestep sizes, and the accumulation of round-off and truncation errors can introduce energy drift over the course of the simulation.

Figures 2a–2d compares total kinetic, potential, and total energy for $N = 32, 64, 128$, and 256. For the smaller systems ($N = 32$ and $N = 64$), the total energy remains relatively stable over time, with only modest fluctuations that do not significantly diverge from the initial values. This indicates that, for these system sizes, our chosen timestep and integration schemes effectively preserve total energy on the timescales examined.

As I increase N to 128, there begin to be more pronounced variability in total energy, though the system still appears to retain rough energy balance. The fluctuations can be attributed to the increasing complexity of gravitational interactions and more frequent close encounters. Nevertheless, the energy still does not exhibit a catastrophic drift away from its initial level.

At $N = 256$, however, energy conservation

begins to break down more noticeably. Over time, the total energy curve drifts significantly, indicating that the numerical errors and collision-related adjustments accumulate faster than they can be mitigated. This behavior points toward the need for more robust integration methods, smaller timesteps, or improved collision handling as N grows large. The rapidly evolving and increasingly crowded environment of a high-N system challenges the stability of the current approach, causing greater deviations from exact energy conservation.

In summary, while total energy is reasonably well conserved for smaller systems, higher particle numbers lead to more substantial energy drift. Future improvements, such as adaptive timestep selection or higher-order symplectic integrators, may be necessary to maintain better energy conservation in large-scale N-body simulations.

IV.III Solver Runtime and OpenMP Scaling

We timed the C++ solvers on an Apple M2 with four performance cores, four efficiency cores, and 8 GiB of RAM. Both executables used Apple Clang 17 with `-O2`. The parallel build linked against LLVM's `libomp`. All solvers used the same seeded initial particles at a given N, with $L = 10$, $G = 1$, $\Delta t = 2^{-17}$, and $\theta = 0.5$ for Barnes–Hut. The benchmark ran four steps and reports the median of three wall times, including initialization and file I/O. Table 1 and Fig. 3 compare solvers. We timed the direct solver through $N = 16{,}384$ and Barnes–Hut through $N = 131{,}072$ with 1, 2, 4, and 8 OpenMP threads.

Table 1: Median wall time in seconds for four C++ simulation steps. A dash denotes a case outside the direct benchmark.

Particles	Direct	Serial Barnes–Hut	OpenMP (4)	OpenMP (8)
1,024	0.0175	0.0136	0.0109	0.0111
4,096	0.2134	0.0518	0.0224	0.0256
16,384	3.1598	0.3313	0.1127	0.0948
65,536	—	1.6341	0.5364	0.4744
131,072	—	5.1946	1.5983	1.6082

At $N = 16{,}384$, serial Barnes–Hut was 9.54× faster than direct summation. Four OpenMP threads gave a further 2.94× speedup, for a combined 28.0× speedup over direct summation. Eight threads reduced the wall time to 0.0948 s.

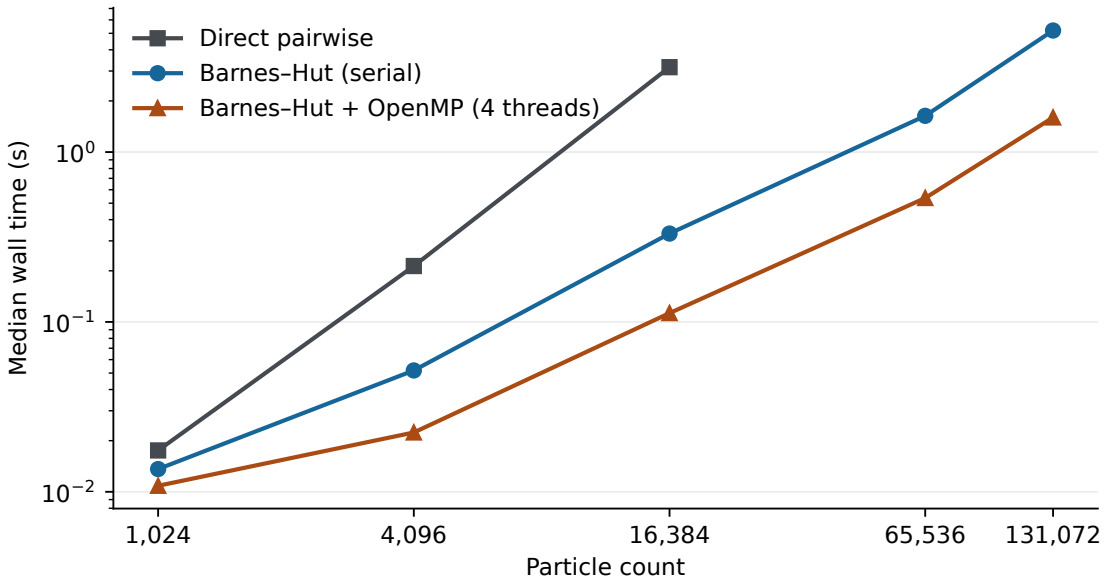


Figure 3: Wall time versus particle count for direct summation, serial Barnes–Hut, and Barnes–Hut with four OpenMP threads. Both axes use a logarithmic scale.

At $N = 131{,}072$, four threads delivered a 3.25× speedup over serial Barnes–Hut and eight threads gave nearly the same wall time (Fig. 4). The remaining serial work and thread overhead limit the speedup of the full simulation.

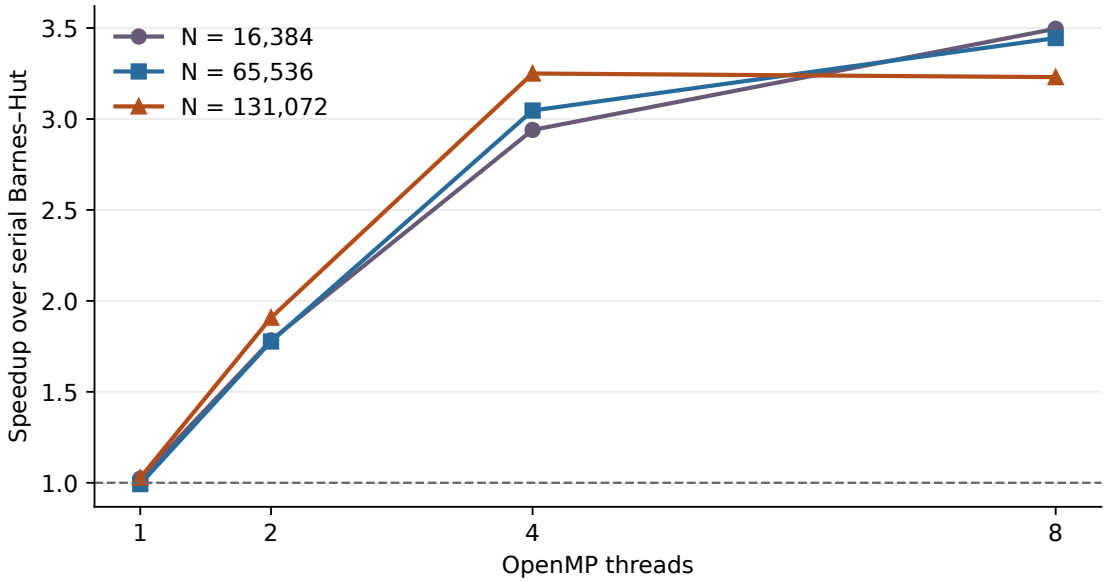


Figure 4: OpenMP speedup for the three largest cases, measured as serial Barnes–Hut wall time divided by threaded wall time. The dashed line marks equal wall time.

Across the four-step comparisons, the largest relative differences between direct and serial Barnes–Hut were 3.73×10^{-7} for kinetic energy, 2.62×10^{-5} for potential energy, and 6.11×10^{-5} for angular momentum magnitude, each measured relative to the serial Barnes–Hut value. OpenMP

and serial Barnes–Hut agreed to within 1.34×10^{-14} in potential energy. Their kinetic energy and angular momentum diagnostics matched in these runs. The parallel reduction changes the summation order, which accounts for the potential energy roundoff. These comparisons cover short runs with the stated initial conditions.

Fortran comparison. In a separate shared-input comparison at $N = 16{,}384$, the four-step Fortran run took 6.14 s with direct summation and 0.52 s with Barnes–Hut. The corresponding C++ times were 3.15 s and 0.32 s. Both serial Barnes–Hut backends completed a one-step run at $N = 1{,}048{,}576$, in about 42 seconds for C++ and 55 seconds for Fortran. The one-step diagnostics agreed within a relative difference of 1.86×10^{-16}. The repository includes the benchmark scripts and the OpenMP timing data.

V Discussion

The results presented in this work highlight both the capabilities and the limitations of our current N-body simulation approach. By examining key physical invariants—particularly total angular momentum and total energy—we gain insights into the accuracy and stability of our chosen numerical methods.

Angular Momentum Conservation

The angular momentum results confirm that, for a wide range of particle numbers ($N = 32$ to 256), the system retains its rotational invariants with only minimal numerical drift. Angular momentum deviations remain on the order of 10^{-9} to 10^{-11}, even though the total angular momentum itself is on the order of 10^{1}. This minuscule relative error suggests that the core integration scheme and collision model effectively preserve rotational characteristics over the simulated timescales. The stability of angular momentum across increasing complexity and particle number is encouraging and points to the robustness of the underlying algorithms used to approximate gravitational forces and handle particle interactions.

Energy Conservation and Scaling Challenges

In contrast, energy conservation proves more challenging as N grows. For small systems ($N = 32$ and $N = 64$), total energy remains relatively stable, indicating that the simple integration scheme and chosen timestep sizes are adequate for these modest scales. As we move to $N = 128$, fluctuations in total energy become more pronounced, yet still manageable, reflecting the increasing complexity and frequency of gravitational interactions and close encounters.

However, by $N = 256$, the simulation struggles to maintain energy conservation. The significant drift observed in total energy suggests

that numerical errors accumulate more rapidly in dense and dynamically active environments. Factors contributing to this drift include:

- **Time integration.** While straightforward integrators like the Euler or low-order schemes are easy to implement, they do not guarantee long-term conservation of energy. The cumulative effect of truncation errors becomes substantial as N increases.
- **Collision Handling:** Although our elastic collision model conserves momentum and kinetic energy in the instantaneous interaction, multiple collisions and close encounters introduce subtle numerical artifacts. Over many timesteps, small inaccuracies can accumulate, contributing to observable energy drift.
- **Parameter Choices:** A fixed timestep Δt may be appropriate for small N but becomes less optimal for larger systems that contain regions of high density and rapidly changing gravitational potentials. Without adaptive refinement, large systems can rapidly develop unstable numerical behaviors.

Implications for Large-Scale Simulations

The observed energy drift at higher N highlights the need for more sophisticated integration strategies when scaling up simulations. Approaches such as symplectic integrators, velocity Verlet, or other higher-order methods can provide more accurate long-term energy conservation. Additionally, adaptive timestep selection—where smaller timesteps are used during close encounters or in dense regions—could mitigate some of the errors that accumulate in large, complex systems.

Beyond integration methods, data structures like the Barnes–Hut tree, which have already proven effective in reducing computational complexity, can be further optimized. More accurate cell-opening criteria or hybrid approaches that combine direct summation for near-neighbors with Barnes–Hut approximations for distant particles might yield improved stability.

The longer conservation runs used up to 256 particles, while the performance tests reached over one million particles for a single step. Measuring long-term energy behavior at the larger sizes will require longer runs and a separate timestep study.

Future Directions

Moving forward, several improvements are possible:

- **Higher-Order and Symplectic Integrators:** Implementing integrators designed to conserve phase-space volume and energy more rigorously over long timescales could yield more stable results, especially

for large N.

- **Adaptive Timestepping:** Dynamically adjusting the timestep in response to the system's local conditions would help resolve close encounters more accurately, reducing the accumulation of numerical errors.
- **Refined Collision Models:** Incorporating more physically realistic collision treatments or implementing methods that minimize unphysical overlapping states could improve overall energy stability.
- **Scaling Tests:** Profiling the serial tree build and collision handling would show which stages to parallelize next. A separate GPU benchmark could measure how the solver scales on a different architecture.

In conclusion, while our simulations demonstrate stable angular momentum conservation and reliable energy behavior for smaller N, the results at higher N emphasize the need for more advanced numerical techniques. By refining our integration schemes, timestepping strategies, and collision handling, I can push toward more accurate, stable, and scalable N-body simulations, ultimately gaining a deeper understanding of gravitational interactions in complex astrophysical environments.

Acknowledgments

This is a final report for a course on the Computer Programming on Geosciences, written by a senior student from the Department of Physics at National Taiwan University. Please note that I have tried my best to eliminate typos and errors. So if there are any, they are unintentional, and I ask for your understanding and forgiveness.

Author Declarations

Conflict of Interest

The authors have no conflicts to disclose.

References

[1] Josh Barnes and Piet Hut. A hierarchical O(N log N) force-calculation algorithm. *Nature*, 324(6096):446–449, December 1986.

[2] OpenMP Architecture Review Board. OpenMP application programming interface, version 5.2. https://www.openmp.org/spec-html/5.2/openmpsu52.html, 2021.

[3] W. Dehnen and J. I. Read. N-body simulations of gravitational dynamics. *The European Physical Journal Plus*, 126(5):55, May 2011.

[4] Lars Nylons, Mark Harris, and Jan Prins. Fast n-body simulation with cuda. *GPU gems*, 3:62–66, 2007.

[5] Sverre J Aarseth. Direct methods for n-body simulations. In *Multiple time scales*, pages 377–418. Elsevier, 1985.

[6] Joachim Gerhard Stadel. *Cosmological N-body simulations and their analysis*. University of Washington, 2001.

[7] Erich Elsen, Vaidyanathan Vishal, Mike Houston, Vijay Pande, Pat Hanrahan, and Eric Darve. N-body simulations on gpus. *arXiv preprint arXiv:0706.3060*, 2007.

[8] Daniel J Eisenstein and Piet Hut. Hop: a new group-finding algorithm for n-body simulations. *The Astrophysical Journal*, 498(1):137, 1998.

[9] Edmund Bertschinger and James M Gelb. Cosmological n-body simulations. *Computers in Physics*, 5:164–175, 1991.

[10] Evangelista Athanassoula. On the nature of bulges in general and of box/peanut bulges in particular: input from n-body simulations. *Monthly Notices of the Royal Astronomical Society*, 358(4):1477–1488, 2005.